

$(t_0 \sigma_0 Q_0 (FS_0 \dots (\text{stack (frame } E[(\text{spawn } v_{\text{coro}})] \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [spawn]
 $(t_1 \sigma_1 Q_1 (FS_0 \dots (\text{stack (frame } E[(\text{task } x_{\text{async}})] \text{ } l) F \dots) FS_1 \dots))$
 $(t_0 \sigma_0 Q_0 (FS_0 \dots (\text{stack (frame } E[(\text{run } v)] \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [task-run]
 $(t_1 \sigma_1 Q_1 (FS_0 \dots (\text{stack (frame } E[(\text{task } x_{\text{async}})] \text{ } l) F \dots) FS_1 \dots))$
 where $(\text{ptr } x_{\text{async}}) = \text{malloc}[\sigma_0]$
 $(t_0 \sigma_0 Q (FS_0 \dots (\text{stack (frame } E[(\text{tagged } x_{\text{running}} \text{ } E_{\text{inner}}[(\text{await (task } x_{\text{async}})])]) \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [await-task-pending]
 $(t_1 \sigma_1 Q (FS_0 \dots (\text{stack (frame } E[(\text{tag } x_{\text{running}})] \text{ } l) F \dots) FS_1 \dots))$
 where $(\text{not in-tag?}[E_{\text{inner}}]), (\text{not task-ready?}[\sigma_0, x_{\text{async}}])$
 $(t_0 \sigma_0 Q (FS_0 \dots (\text{stack (frame } E[(\text{await (task } x_{\text{async}})]) \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [task-await-done]
 $(t_1 \sigma_1 Q (FS_0 \dots (\text{stack (frame } E[v] \text{ } l) F \dots) FS_1 \dots))$
 where $(\text{done } v) = \text{task-status}[v_{\text{obj}}]$
 $(t_0 \sigma_0 Q (FS_0 \dots (\text{stack (frame } E[(\text{await (task } x_{\text{async}})]) \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [task-await-failed]
 $(t_1 \sigma_1 Q (FS_0 \dots (\text{stack (frame } E[(\text{throw } v)] \text{ } l) F \dots) FS_1 \dots))$
 where $v_{\text{obj}} = \text{lookup}[\sigma_0, x_{\text{async}}], (\text{failed } v) = \text{task-status}[v_{\text{obj}}]$
 $(t_0 \sigma Q_0 (FS_0 \dots (\text{stack (frame } v_{\text{coro}} \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [task-reschedule]
 $(t_1 \sigma Q_1 (FS_0 \dots (\text{stack } F \dots) FS_1 \dots))$
 where $(\text{async? } l), x_{\text{async}} = l, v_{\text{obj}} = \text{lookup}[\sigma, x_{\text{async}}], \text{task-coro-eq?}[v_{\text{obj}}, v_{\text{coro}}], (\text{pending } _ \dots) = \text{task-status}[v_{\text{obj}}]$
 $(t_0 \sigma_0 Q_0 (FS_0 \dots (\text{stack (frame } v_{\text{coro}} \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [task-reschedule-cancelled]
 $(t_1 \sigma_1 Q_1 (FS_0 \dots (\text{stack } F \dots) FS_1 \dots))$
 where $(\text{async? } l), x_{\text{async}} = l, v_{\text{obj}} = \text{lookup}[\sigma_0, x_{\text{async}}], \text{task-coro-eq?}[v_{\text{obj}}, v_{\text{coro}}], \text{cancelled} = \text{task-status}[v_{\text{obj}}]$
 $(t_0 \sigma_0 Q (FS_0 \dots (\text{stack (frame } v \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [task-return]
 $(t_1 \sigma_1 Q (FS_0 \dots (\text{stack } F \dots) FS_1 \dots))$
 where $(\text{async? } l), x_{\text{async}} = l, v_{\text{obj}} = \text{lookup}[\sigma_0, x_{\text{async}}], (\text{not task-coro-eq?}[v_{\text{obj}}, v])$
 $(t_0 \sigma_0 Q (FS_0 \dots (\text{stack (frame } E[(\text{throw } v)] \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [async-throw]
 $(t_1 \sigma_1 Q (FS_0 \dots (\text{stack } F \dots) FS_1 \dots))$
 where $(\text{async? } l), (\text{not in-handler?}/\text{aio}[E]), x_{\text{async}} = l$
 $(t_0 \sigma_0 Q (FS_0 \dots (\text{stack (frame } E[(\text{cancel (task } x_{\text{async}})]) \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [cancel]
 $(t_1 \sigma_1 Q (FS_0 \dots (\text{stack (frame } E[(\text{void})] \text{ } l) F \dots) FS_1 \dots))$
 where $v_{\text{obj}} = \text{lookup}[\sigma_0, x_{\text{async}}]$
 $(t_0 \sigma_0 Q (FS_0 \dots (\text{stack (frame } E[(\text{os/io natural } v)] \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [os/io]
 $(t_1 \sigma_1 Q (FS_0 \dots (\text{stack (frame } E[(\text{spawn (tag } x_{\text{tag}})] \text{ } l) F \dots) FS_1 \dots))$
 where $(x_{\text{dummy}} x_{\text{tag}}) = \text{lib:gensyms}[\sigma_0, \sigma_0], \sigma_1 = \text{ext1}[\sigma_0, (x_{\text{tag}} (\text{coroutine (lambda (x_{\text{dummy}})$
 $\text{lib:begin}[\sigma_{\text{dummy}},$
 $\text{lib:while}[\leq (\text{os/time}) \text{ lib:}\Sigma[t_0, \text{natural}],$
 $(\text{yield (tag } x_{\text{tag}})]),$
 $v)))]$
 $(t_0 \sigma Q (FS_0 \dots (\text{stack (frame } E[(\text{os/time})] \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [os/time]
 $(t_1 \sigma Q (FS_0 \dots (\text{stack (frame } E[t_0] \text{ } l) F \dots) FS_1 \dots))$
 $(t_0 \sigma Q_0 (FS_{\text{main}} FS_0 \dots (\text{stack } FS_1 \dots)) \longrightarrow$ [thread-work-steal]
 $(t_1 \sigma Q_1 (FS_{\text{main}} FS_0 \dots (\text{stack } F_{\text{head}}) FS_1 \dots))$
 where $(F_{\text{head}} Q_1) = \text{q-pop}[Q_0], \text{all-busy?}[FS_0, \dots], t_1 = \text{step}[t_0]$
 $(t_0 \sigma_0 Q (FS_0 \dots (\text{stack (frame } e_0 \text{ } l) F \dots) FS_1 \dots)) \longrightarrow$ [base-step]
 $(t_1 \sigma_1 Q (FS_0 \dots (\text{stack (frame } e_1 \text{ } l) F \dots) FS_1 \dots))$
 where $(\text{not value?}[e]), (\sigma_1 e_1) = \Downarrow \text{base}[\sigma_0, e_0], t_1 = \text{step}[t_0]$
 $(t_0 \sigma Q ((\text{stack (frame } E[(\text{block (tag } x_{\text{coro}})]) \text{ } l) FS_1 \dots)) \longrightarrow$ [block]
 $(t_1 \sigma Q ((\text{stack (frame } E[(\text{block (spawn (tag } x_{\text{coro}})]) \text{ } l) FS_1 \dots))$
 $(t_0 \sigma () ((\text{stack (frame } E[(\text{block (task } x_{\text{async}})]) \text{ } l) FS_{\text{rest}} \dots)) \longrightarrow$ [unblock-value]
 $(t_1 \sigma () ((\text{stack (frame } E[v] \text{ } l) FS_{\text{rest}} \dots))$
 where $(\text{done } v) = \text{task-status}[\text{lookup}[\sigma, x_{\text{async}}]], (\text{not any-busy?}[FS_{\text{rest}}, \dots]), \text{none} = \text{find-unawaited-error}[\sigma]$
 $(t_0 \sigma () ((\text{stack (frame } E[(\text{block (task } x_{\text{async}})]) \text{ } l) FS_{\text{rest}} \dots)) \longrightarrow$ [unblock-error]
 $(t_1 \sigma () ((\text{stack (frame } E[(\text{throw } v)] \text{ } l) FS_{\text{rest}} \dots))$
 where $(\text{not any-busy?}[FS_{\text{rest}}, \dots]), (\text{some } v) = \text{find-unawaited-error}[\sigma]$